

# vuln\_lkm.pdf

---

## *How the Vuln-LKM loadable kernel module works*

Repository: [github.com/mayday5000/Vuln-LKM](https://github.com/mayday5000/Vuln-LKM)

Device node: /dev/vuln\_lkm

Module: vuln\_lkm.ko (misc character driver)

Document generated from the committed source

*Educational material for the Vuln-LKM loadable kernel module. Not an exploit guide. Integer wrap and a bounded-struct string copy are described as they appear in the source.*

# 1. What this object is

vuln\_lkm is a Linux Loadable Kernel Module (LKM). That sentence has two claims that are easy to blur. First, it is kernel code: after insmod it shares the kernel address space, can call printk and copy\_from\_user, and a bug in it can panic the machine. Second, it is loadable: it is not linked into vmlinux. You build a .ko against the headers of the kernel that will load it, then insert it at runtime. rmmod tears it down.

It is also a driver, in the narrow character-device sense. It is not a PCI driver, not a platform driver, not a USB driver, not a block driver. It does not bind to hardware. It registers a miscdevice named vuln\_lkm. The kernel then creates /dev/vuln\_lkm. Userspace opens that node and talks through ioctl. That is the entire I/O surface.

The module exists as a lab. Three operations are intentionally wrong: unsigned 32-bit addition without a saturation check, unsigned 32-bit subtraction without an underflow check, and memcpy into a 32-byte kernel stack buffer using a length that is only capped at 256 (the ioctl struct), not at 32. GET and SET are the honest controls so you can read and plant g\_val before ADD or SUB.

## 1.1 LKM versus built-in driver

A built-in driver is compiled into vmlinux (or into a vmlinux that is later compressed to bzImage). Its initcall runs as the kernel boots. You cannot rmmod it. An LKM is a relocatable ELF object. The module loader (kernel/module.c) applies relocations, resolves exported symbols (printk, copy\_from\_user, misc\_register, ...), calls the function marked module\_init, and keeps a struct module around for sysfs and rmmod.

kbld for an out-of-tree module is the Makefile line obj-m += vuln\_lkm.o plus make -C /lib/modules/\$(uname -r)/build M=\$PWD modules. The running kernel's build tree supplies the Module.symvers, the compiler flags, and the vermagic string. If vermagic does not match, insmod refuses. That is why WSL Ubuntu 20.04 must use that distro's linux-headers package, not a random mainline tree.

## 1.2 Why miscdevice

Linux character drivers historically called register\_chrdev or cdev\_init plus class\_create plus device\_create, and picked a major number. miscdevice is a shortcut for 'one minor under the shared misc major (10)'. MISC\_DYNAMIC\_MINOR asks the core to pick a free minor. The name field becomes the /dev node via the misc class. mode = 0666 in our struct is an explicit lab choice so the CLI can open the node without being root after insmod; a production driver would not do that.

The only file\_operations we fill are owner and unlocked\_ioctl. There is no .read, no .write, no .mmap. If you cat the node you get -EINVAL from the default. All lab traffic is ioctl. We use unlocked\_ioctl, not the old .ioctl which used to run under the Big Kernel Lock. unlocked\_ioctl is the post-2.6.11 path: the VFS calls it without the BKL, and the driver is responsible for its own serialization. vuln\_lkm has none: g\_val is a naked static u32. Concurrent ioctls can tear it. The lab ignores that on purpose.

# 2. Shared ABI: vuln\_lkm.h

The header is compiled both in the kernel (with linux/ioctl.h and linux/types.h) and in the CLI (with sys/ioctl.h and stdint.h). That dual include is the whole point of a UAPI-style header: the ioctl numbers and the struct layouts must match on both sides of copy\_from\_user.

```
#ifndef VULN_LKM_H
#define VULN_LKM_H

#ifdef __KERNEL__
#include <linux/ioctl.h>
#include <linux/types.h>
#else
#include <sys/ioctl.h>
#include <stdint.h>

```

```

typedef uint32_t __u32;
#endif

#define VULN_LKM_NAME "vuln_lkm"
#define VULN_LKM_IOC_MAGIC 0x1B
#define VULN_LKM_STR_MAX 256
#define VULN_LKM_KBUF 32

struct vuln_lkm_int {
    __u32 value;
};

struct vuln_lkm_str {
    __u32 len;
    char data[VULN_LKM_STR_MAX];
};

#define VULN_LKM_GET _IOR(VULN_LKM_IOC_MAGIC, 1, struct vuln_lkm_int)
#define VULN_LKM_SET _IOW(VULN_LKM_IOC_MAGIC, 2, struct vuln_lkm_int)
#define VULN_LKM_ADD _IOW(VULN_LKM_IOC_MAGIC, 3, struct vuln_lkm_int)
#define VULN_LKM_SUB _IOW(VULN_LKM_IOC_MAGIC, 4, struct vuln_lkm_int)
#define VULN_LKM_STR _IOW(VULN_LKM_IOC_MAGIC, 5, struct vuln_lkm_str)

#endif

```

VULN\_LKM\_IOC\_MAGIC is 0x1B. The \_IOR/\_IOW macros encode direction, type, ordinal, and sizeof the third argument into a 32-bit command word. Userspace and kernel must agree on that word. A mismatch yields -ENOTTY from the default branch.

struct vuln\_lkm\_int is a single \_\_u32. On this ABI it is 4 bytes with no padding. struct vuln\_lkm\_str is \_\_u32 len plus char data[256]: 260 bytes. The kernel stack buffer kbuf is NOT 256 bytes. It is VULN\_LKM\_KBUF, 32. That size split is the string bug.

## 2.1 ioctl command encoding

On Linux, \_IOC(dir,type,nr,size) packs: nr in bits 0–7, type in 8–15, size in 16–29, dir in 30–31. \_IOC\_WRITE is 1, \_IOC\_READ is 2. So VULN\_LKM\_GET = \_IOR(0x1B, 1, struct vuln\_lkm\_int) is a read of 4 bytes, ordinal 1. VULN\_LKM\_ADD is a write of 4 bytes, ordinal 3. VULN\_LKM\_STR is a write of 260 bytes, ordinal 5. The CLI prints cmd in hex on every call so you can confirm the packing on your libc.

The third argument of ioctl(2) is passed to the driver as unsigned long arg. For these commands it is a user pointer to the struct. The kernel must copy, not dereference arg as a kernel pointer. GET/SET/ADD/SUB all go through copy\_from\_user or copy\_to\_user. STR copies the whole vuln\_lkm\_str, then memcpy from the kernel-side copy into kbuf — and that second copy is where the length is trusted too far.

## 3. Module lifetime

vuln\_lkm\_init is module\_init. It calls misc\_register(&vuln\_lkm\_dev). On success the kernel has a live misc device and printk announces /dev/vuln\_lkm and g\_val=100. On failure it returns the errno and the module does not stay loaded. vuln\_lkm\_exit is module\_exit: misc\_deregister. There is no kfree of lab state because g\_val is static and STR's kbuf is on-stack per call. Nothing is left on the heap.

g\_val starts at 100. That is the only persistent lab state. It lives in .bss/.data of the module. After rmmod the pages are gone. A second insmod starts at 100 again.

## 4. The ioctl switch

vuln\_lkm\_ioctl(filp, cmd, arg) is the single entry. filp is unused. cmd selects a case. Unknown cmd returns -ENOTTY. Each known case is a straight-line function so GDB can break on vuln\_lkm\_add, vuln\_lkm\_sub, vuln\_lkm\_str, or the dispatcher itself.

## 4.1 GET — the honest read

GET fills a local `vuln_lkm_int` with `g_val` and `copy_to_user`'s it. If the user pointer is bad, `-EFAULT`. `printk` logs the value. This is how the CLI proves wrap after ADD/SUB. There is no bug here.

## 4.2 SET — the honest write

SET `copy_from_user`'s a `u32` into `req`, `printk`'s the transition, then stores `g_val = req.value`. Use SET to plant 10 before SUB 11, or `0xffffffff` before ADD 3. No check, but no wrap either: it is a store, not an add.

## 4.3 ADD — integer overflow (wrap)

Path: CLI add N  $\rightarrow$  `ioctl(VULN_LKM_ADD)`  $\rightarrow$  `copy_int_from_user`  $\rightarrow$  `vuln_lkm_add(n)`  $\rightarrow$  `g_val = g_val + n`. The add is `u32 + u32` in C. Unsigned overflow is defined as modulo  $2^{32}$ . There is no if (`n > U32_MAX - g_val`) return `-EINVAL`. `printk` reports `wrap=yes` when the result is less than before, which is the modular wrap detector for addition.

Example from the CLI docs: SET `0xffffffff`, ADD 3.  $4294967294 + 3 = 4294967297 \equiv 1 \pmod{2^{32}}$ . `dmesg`: ADD  $4294967294 + 3 = 1$  (`wrap=yes`). This is not undefined behavior. It is defined wrap. The 'vulnerability' in the lab sense is that a length or index derived from this value would be wrong; here the value is the lab artifact itself.

## 4.4 SUB — integer underflow (wrap)

Path: CLI sub N  $\rightarrow$  `ioctl(VULN_LKM_SUB)`  $\rightarrow$  `copy_int_from_user`  $\rightarrow$  `vuln_lkm_sub(n)`  $\rightarrow$  `g_val = g_val - n`. No if (`n > g_val`). Unsigned subtraction also modulo  $2^{32}$ . SET 10, SUB 11 yields 4294967295. `printk` `wrap=yes` when `g_val >` before.

## 4.5 STR — string buffer overflow

Path: CLI str S  $\rightarrow$  fill `vuln_lkm_str {len=strlen(S), data=S}`  $\rightarrow$  `ioctl(VULN_LKM_STR)`  $\rightarrow$  `vuln_lkm_str(arg)`. First `copy_from_user` copies the whole 260-byte struct into `req` on the kernel stack. Then if (`req.len > 256`) `req.len = 256`. That cap is against the `ioctl` blob, not against `kbuf`. `kbuf` is `char kbuf[32]`. Then `memcpy(kbuf, req.data, req.len)`. If `req.len` is 40, 40 bytes are written into 32 bytes of stack.

What sits after `kbuf` is compiler-dependent (canary, saved registers, return address, the rest of `vuln_lkm_str`'s `req`). This lab does not teach how to turn that into privilege escalation. It teaches that the length used by `memcpy` was not the length of the destination. Run STR tests in QEMU. A long enough copy can panic the guest.

# 5. copy\_from\_user and why arg is not a kernel pointer

unsigned long `arg` in `ioctl` is a userspace address when the command takes a pointer. Dereferencing it as a kernel pointer is a bug (the original HEVD-style practice driver even had a `buffer_overflow` that did `strlen` on a user pointer). `vuln_lkm` copies first. `copy_from_user` uses `access_ok` and an architecture-specific copier that handles faults by returning a remaining-byte count. A nonzero return becomes `-EFAULT` in our helpers.

STR's `memcpy` after the copy is kernel-to-kernel. There is no further user-fault handling. A too-large `len` smashes kernel stack, not user stack.

# 6. Concurrency, context, and what this driver does not do

`ioctl` runs in process context of the calling task. It may block (`copy_from_user` can sleep). You are not in a hardirq. You are not at Windows IRQL `DISPATCH_LEVEL`. You may not call this code from an atomic section that forbids sleep. There is no spinlock around `g_val`. Two threads adding at once can lose updates. The lab is single-threaded in the CLI.

There is no `compat_ioctl`. A 32-bit userspace on a 64-bit kernel with a struct that contained pointers would be wrong; our structs are fixed-width integers and a char array, so they happen to match. There is no `.read/.write`. There is

no sysfs knob besides the automatic `/sys/module/vuln_lkm/` tree. There is no udev extra rule; misc handles the node.

## 7. Complete source listing: vuln\_lkm.c

The file as committed. GDB breakpoints are intended on `vuln_lkm_add`, `vuln_lkm_sub`, `vuln_lkm_str`, and `vuln_lkm_ioctl`.

```
#include <linux/module.h>
#include <linux/miscdevice.h>
#include <linux/uaccess.h>

#include "vuln_lkm.h"

static u32 g_val = 100;

static int copy_int_from_user(unsigned long arg, struct vuln_lkm_int *out)
{
    if (copy_from_user(out, (void __user *)arg, sizeof(*out)))
        return -EFAULT;
    return 0;
}

/* GDB: break vuln_lkm_add */
static int vuln_lkm_add(u32 n)
{
    u32 before = g_val;

    /* No check against U32_MAX - n. The add wraps. */
    g_val = g_val + n;
    printk(KERN_INFO "vuln_lkm: ADD %u + %u = %u (wrap=%s)\n",
           before, n, g_val, (g_val < before) ? "yes" : "no");
    return 0;
}

/* GDB: break vuln_lkm_sub */
static int vuln_lkm_sub(u32 n)
{
    u32 before = g_val;

    /* No check against n > g_val. The sub wraps. */
    g_val = g_val - n;
    printk(KERN_INFO "vuln_lkm: SUB %u - %u = %u (wrap=%s)\n",
           before, n, g_val, (g_val > before) ? "yes" : "no");
    return 0;
}

/* GDB: break vuln_lkm_str */
static int vuln_lkm_str(unsigned long arg)
{
    struct vuln_lkm_str req;
    char kbuf[VULN_LKM_KBUF];

    if (copy_from_user(&req, (void __user *)arg, sizeof(req)))
        return -EFAULT;

    if (req.len > VULN_LKM_STR_MAX)
        req.len = VULN_LKM_STR_MAX;

    memset(kbuf, 0, sizeof(kbuf));
    printk(KERN_INFO "vuln_lkm: STR len=%u into kbuf[%u]\n",
           req.len, VULN_LKM_KBUF);
    /*
     * kbuf is 32 bytes. req.len is not compared to VULN_LKM_KBUF.
     * memcpy uses the user-supplied length as-is.
     */
    memcpy(kbuf, req.data, req.len);
    return 0;
}
```

```

static long vuln_lkm_ioctl(struct file *filp, unsigned int cmd, unsigned long arg)
{
    struct vuln_lkm_int req = {0};

    switch (cmd) {
        case VULN_LKM_GET:
            req.value = g_val;
            if (copy_to_user((void __user *)arg, &req, sizeof(req)))
                return -EFAULT;
            printk(KERN_INFO "vuln_lkm: GET %u (0x%x)\n", g_val, g_val);
            return 0;

        case VULN_LKM_SET:
            if (copy_int_from_user(arg, &req))
                return -EFAULT;
            printk(KERN_INFO "vuln_lkm: SET %u -> %u\n", g_val, req.value);
            g_val = req.value;
            return 0;

        case VULN_LKM_ADD:
            if (copy_int_from_user(arg, &req))
                return -EFAULT;
            return vuln_lkm_add(req.value);

        case VULN_LKM_SUB:
            if (copy_int_from_user(arg, &req))
                return -EFAULT;
            return vuln_lkm_sub(req.value);

        case VULN_LKM_STR:
            return vuln_lkm_str(arg);

        default:
            return -ENOTTY;
    }
}

static const struct file_operations vuln_lkm_fops = {
    .owner = THIS_MODULE,
    .unlocked_ioctl = vuln_lkm_ioctl,
};

static struct miscdevice vuln_lkm_dev = {
    .minor = MISC_DYNAMIC_MINOR,
    .name = VULN_LKM_NAME,
    .fops = &vuln_lkm_fops,
    .mode = 0666,
};

static int __init vuln_lkm_init(void)
{
    int ret = misc_register(&vuln_lkm_dev);

    if (ret)
        return ret;
    printk(KERN_INFO "vuln_lkm: loaded, /dev/%s, g_val=%u\n",
           VULN_LKM_NAME, g_val);
    return 0;
}

static void __exit vuln_lkm_exit(void)
{
    misc_deregister(&vuln_lkm_dev);
    printk(KERN_INFO "vuln_lkm: unloaded\n");
}

module_init(vuln_lkm_init);
module_exit(vuln_lkm_exit);
MODULE_LICENSE("GPL");
MODULE_AUTHOR("Damian");
MODULE_DESCRIPTION("Educational LKM: integer wrap and string buffer overflow");

```

## 8. Build, load, unload

make module runs kbuild. make user builds the CLI. load.sh rmmmods any old name (vuln\_lkm, mini\_vuln, int\_bounds), insmod vuln\_lkm.ko, chmod 666 /dev/vuln\_lkm. unload.sh rmmmods. dmesg is the kernel-side log for GET/SET/ADD/SUB/STR.

If insmod fails with Invalid module format, the .ko was built against a different kernel than uname -r. Rebuild on that WSL kernel. If /dev/vuln\_lkm is missing, misc\_register failed or udev/devtmpfs did not create the node; dmesg will say.

## 9. Mapping lab commands to functions

- vuln\_lkm\_cli get → VULN\_LKM\_GET → vuln\_lkm\_ioctl GET branch
- vuln\_lkm\_cli set N → VULN\_LKM\_SET → store g\_val
- vuln\_lkm\_cli add N → VULN\_LKM\_ADD → vuln\_lkm\_add (wrap on overflow)
- vuln\_lkm\_cli sub N → VULN\_LKM\_SUB → vuln\_lkm\_sub (wrap on underflow)
- vuln\_lkm\_cli str S → VULN\_LKM\_STR → vuln\_lkm\_str (memcpy into 32-byte kbuf)

See vuln\_lkm\_cli.pdf for the userspace side and vuln\_lkm\_run\_debug.pdf for QEMU, GDB, ioctl/IRQL/character-device theory, and the 40-page lab manual.